GEMPRO Notebook - Extended

GEMPRO

Índice

1 Contest **3**
- 1.1 template.cpp . . . 3
- 1.2 .bashrc . . . 3
- 1.3 Template de stress testing . . . 3
- 1.4 .vimrc . . . 4

2 Matemática **4**
- 2.1 Algoritmo Euclideano . . . 4
- 2.2 Crivo . . . 4
- 2.3 CRT . . . 4
- 2.4 Teste de primalidade . . . 5

3 Estruturas de Dados **5**
- 3.1 Fenwick Tree . . . 5
- 3.2 Segtree . . . 5
- 3.3 Lazy Segtree . . . 6
- 3.4 Sparse Table . . . 7
- 3.5 Disjoint Set Union . . . 7
- 3.6 Convolução . . . 7
  - 3.6.1 FFT . . . 7
  - 3.6.2 NTT . . . 8
- 3.7 Max Add Segtree . . . 9
- 3.8 Zeta e Mobius . . . 9
- 3.9 Árvore de Li Chao . . . 10
- 3.10 Árvore de Li Chao++ . . . 10
- 3.11 Árvore Cartesiana . . . 11
- 3.12 Árvore de Busca Binária Aleatória . . . 11
- 3.13 Treap implícita . . . 12
- 3.14 Treap de conjunto ordenado . . . 13

4 Grafos **14**
- 4.1 Caminho Euleriano . . . 14
  - 4.1.1 Encontrando qualquer caminho euleriano . . . 15
- 4.2 Max Flow . . . 15
  - 4.2.1 Dinic . . . 15
- 4.3 Min Cost Flow . . . 16
  - 4.3.1 MCMF (SPFA) . . . 16

5 Árvores **17**
- 5.1 Base . . . 17
- 5.2 Movimento em árvores . . . 17
- 5.3 HLD . . . 18
- 5.4 LCA . . . 18
- 5.5 Tree Matching . . . 19

6 Strings **19**
- 6.1 Suffix Array . . . 19
- 6.2 Maior prefixo comum (LCP) . . . 19
- 6.3 KMP . . . 20

7 Geometria **20**
- 7.1 Base . . . 20
- 7.2 Interseção de retas . . . 20
- 7.3 Interseção de segmentos . . . 21
- 7.4 Área de um polígono . . . 21
- 7.5 Verificar se um ponto está dentro de um polígono . . . 21
- 7.6 Convex hull . . . 21
- 7.7 Menor distância euclideana . . . 22

7.8 Interseção de círculo e reta 22
7.9 Interseção de círculos 22
7.10 Tangentes comuns a dois círculos 23
7.11 Soma Minkowski de polígonos convexos 23
7.12 Convex hull online 23
7.13 Corte de Polígono 24

A Fórmulas Úteis **24**
A.1 Contagem e Probabilidade 24
A.2 Somas 25

B Teoremas e Fatos **25**

C Ideias **28**
C.1 Gerais 28
C.2 Calculando somas e integrais 28
C.3 Problemas de Otimização 29

D Debugging **29**
D.1 Geral 29
D.2 WA - Wrong Answer 29
D.3 RE - Runtime Error 29

Contest

template.cpp

Template base de código para abrir um problema, com aliases e utilitários recorrentes na equipe.

```cpp
#include <bits/stdc++.h>

using namespace std;

using ll = long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;

#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
#define rep(i,a,b) for (int i = (a); i < (b); ++i)

mt19937 rng(random_device{}());

int main() {
    cin.tie(0)->sync_with_stdio(0);
}
```

.bashrc

Atalhos e funções de shell para compilar, rodar testes e comparar saídas rapidamente durante a prova.

```bash
c() {
    g++ $1.cpp -o $1 -std=c++20
}
cs() {
    g++ $1.cpp -o $1 -std=c++20
    ↪  -fsanitize=address,undefined,signed-integer-overflow -ggdb
}
rt() { python3 $3.py >t.in; ./$1 <t.in >t.out; ./$2 <t.in >t.ans; }
st() {
    for i in {1..100}; do
        rt $@
        cmp -s t.out t.ans || { echo WA; return 1; }
    done
    echo AC
}
```

Template de stress testing

Gerador base para stress testing local. Use junto com um brute e um solucionador candidato.

```python
from random import randrange as rr, randint as ri, shuffle as sh, sample as
↪  sm
from string import ascii_lowercase as alpha
from itertools import combinations as combs

def out(*a): print(" ".join(map(str, a)))

def outa(a): print(" ".join(map(str, a)))

def tree(n): return [[rr(1, i), i] for i in range(2, n + 1)]

def graph(n, m): return sm(list(combs(range(1, n + 1), 2)), m)

def arr(n, l = 0, r = 10 ** 9): return [ri(l, r) for i in range(n)]

def darr(n, l = 0, r = 10 ** 9): return sm(range(l, r), n)

def perm(n): p = list(range(1, n + 1)); sh(p); return p
```

.vimrc

Configuração enxuta do Vim para edição rápida no ambiente de contest.

```
syntax on
set cin ts=4 sw=4 udf cul nu is

ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \| md5sum \| cut -c
↪  -6
com! F w | call system('clang-format -i ' . expand('%')) | e
```

Matemática

Algoritmo Euclideano

Encontra inteiros x, y e g tais que $ax + by = g$ e $g = gcd(a, b)$. Útil para calcular inversos módulo M. Complexidade: $O(\log(\min(a, b)))$.

b4f64d - math/egcd.cpp - 9 lines

```
ll egcd(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    ll g = egcd(b, a % b, y, x);
    y -= (a / b) * x;
    return g;
} // b4f64d
```

Crivo

Encontra todos os números primos menores que n. Além disso, também encontra para cada inteiro i entre 0 e n, qual o menor fator primo de i. Complexidade: $O(n)$.

eb3eff - math/sieve.cpp - 16 lines

```
struct Sieve {
    vi primes, spf;
    Sieve(int n): spf(n) {
        rep (i, 2, n) {
            if (!spf[i]) {
                spf[i] = i;
                primes.push_back(i);
            }
            for (int j: primes) {
                if (j * i >= n) break;
                spf[i * j] = j;
                if (j == spf[i]) break;
            }
        }
    }
};
```

CRT

Combina duas congruências $x \equiv r_1 \pmod{m_1}$ e $x \equiv r_2 \pmod{m_2}$ em $x \equiv r \pmod{\text{lcm}(m_1, m_2)}$, ou retorna $\{0, -1\}$ se incompatível. Chame em cadeia para combinar mais congruências. Requer egcd. Complexidade: $O(\log(\min(m_1, m_2)))$.

6e0e1c - math/crt.cpp - 12 lines

```
// x = r1 (mod m1), x = r2 (mod m2) -> x = r (mod m)
// returns {r, m} or {0, -1} if incompatible
// chain calls to combine more congruences
pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2) {
    ll x, y, g = egcd(m1, m2, x, y);
    if ((r2 - r1) % g) return {0, -1};
    ll k = m2 / g, m = m1 / g * m2;
    ll q = ((r2 - r1) / g % k + k) % k;
    x = ((x % k) + k) % k;
    ll r = r1 + (__int128)m1 * (q * x % k);
    return {(r % m + m) % m, m};
} // 6e0e1c
```

Teste de primalidade

Verificar se um número é primo em $O(\log(N))$.

478100 - math/primality.cpp - 34 lines

```
using i128 = __int128;
ll powm(ll x, ll e, ll mod) {
    ll r = 1;
    while (e) {
        if (e & 1) r = i128(r)* x % mod;
        e >>= 1;
        x = i128(x) * x % mod;
    }
    return r;
} // 802609

bool checkPrime(ll p, ll a) {
    ll k = p - 1, d = 1;
    while (~k & 1) k >>= 1;
    for (ll e = k; e; e >>= 1) {
        if (e & 1) d = i128(d) * a % p;
        a = i128(a) * a % p;
    }
    if (d == 1 || d == p - 1) return true;
    while ((k <<= 1) < p - 1) {
        d = i128(d) * d % p;
        if (d == p - 1) return true;
    }
    return false;
} // d5cf5e

bool isPrime(ll p) {
    if (p == 1) return false;
    for (int i: {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (p == i) return true;
        if (!checkPrime(p, i)) return false;
    }
    return true;
} // a137c8
```

Estruturas de Dados

Fenwick Tree

Atua com um array de inteiros $a_0, a_1, \ldots, a_{n-1}$. Suporta dois tipos de operação:

- $add(i, val)$: adiciona o valor val ao índice i ($a_i \to a_i + val$). Complexidade: $O(\log(n))$.
- $sum(L, R)$: Calcula a soma $a_L + a_{L+1} + \cdots + a_{R-1}$. Complexidade: $O(\log(n))$.

281281 - dsa/fenwick-tree.cpp - 14 lines

```
struct Fenwick {
    vector<ll> t;
    int n;
    Fenwick(int N) : t(N), n(N) {}
    void add(int i, ll val) {
        for (i++; i <= n; i += i & -i) t[i - 1] += val;
    } // 638501
    ll sum(int r) {
        ll s = 0;
        for (; r; r -= r & -r) s += t[r - 1];
        return s;
    } // 025015
    ll sum(int l, int r) { return sum(r) - sum(l); }
};
```

Segtree

Atua com uma operação $\oplus$ em elementos de uma monoide S com elemento neutro e, em um array $a_1, a_2, \ldots, a_{n-1}$ com $a_i \in S$. Suporta dois tipos de operações:

- $set(i, val)$: modifica o array na posição i ($a_i \to val$). Complexidade: $O(\log(n))$.
- $sum(L, R)$: calcula $a_l \oplus a_{l+1} \oplus \cdots \oplus a_{r-1}$. Complexidade: $O(\log(n))$.

b76e87 - dsa/segtree.cpp - 18 lines

```cpp
template<class S, S (*op)(S, S), S (*e)()>
struct Segtree {
    vector<S> t;
    int n;
    Segtree(int N): t(2 * N, e()), n(N) {}
    void set(int i, S value) {
        t[i += n] = value;
        for (i >>= 1; i; i >>= 1) t[i] = op(t[i << 1], t[i << 1 | 1]);
    } // 630e57
    S query(int l, int r) {
        S al = e(), ar = e();
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) al = op(al, t[l++]);
            if (r & 1) ar = op(t[--r], ar);
        }
        return op(al, ar);
    } // 9ee903
};
```

Lazy Segtree

Similar a uma segtree normal, mas suporta modificações em range. Suporta três tipos de operações:

- $set(i, val)$: modifica o array na posição i ($a_i \to val$). Complexidade: $O(\log(n))$.
- $sum(L, R)$: calcula $a_l \oplus a_{l+1} \oplus \cdots \oplus a_{r-1}$. Complexidade: $O(\log(n))$.
- $apply(L, R, U)$: Aplica a função $mapping$ no range $[L, R)$ ($a_i \to mapping(U, a_i)$, para $i = L, L+1, \ldots, R-1$). Complexidade: $O(\log(n))$.

4ea409 - dsa/lazy-segtree.cpp - 51 lines

```cpp
template<class S, S (*op)(S, S), S (*e)(), class U, S (*mapping)(U, S), U
↪  (*compose)(U, U), U (*id)()>
struct LazySegtree {
    vector<S> t;
    vector<U> d;
    int n;
    LazySegtree(int N): t(4 * N, e()), d(2 * N, id()), n(N) {}
    void push(int i) {
        if (i < n) {
            d[i << 1] = compose(d[i], d[i << 1]);
            d[i << 1 | 1] = compose(d[i], d[i << 1 | 1]);
        }
        if (i < 2 * n) {
            t[i << 1] = mapping(d[i], t[i << 1]);
            t[i << 1 | 1] = mapping(d[i], t[i << 1 | 1]);
            d[i] = id();
        }
    } // 722677
    void set(int j, S val) { set(j, val, 0, n, 1); }
    void set(int j, S val, int tl, int tr, int i) {
        push(i);
        if (tl + 1 == tr) {
            t[i] = val;
            return;
        }
        int tm = (tl + tr) / 2;
        if (j < tm) set(j, val, tl, tm, i << 1);
        else set(j, val, tm, tr, i << 1 | 1);
        t[i] = op(t[i << 1], t[i << 1 | 1]);
    } // eadaef
    void apply(int l, int r, U u) { apply(l, r, u, 0, n, 1); }
    void apply(int l, int r, U u, int tl, int tr, int i) {
        push(i);
        if (r <= tl || tr <= l) return;
        if (l <= tl && tr <= r) {
            t[i] = mapping(u, t[i]);
            if (i < 2 * n) d[i] = compose(u, d[i]);
            return;
        }
        int tm = (tl + tr) / 2;
        apply(l, r, u, tl, tm, i << 1);
        apply(l, r, u, tm, tr, i << 1 | 1);
    } // 630c93
    S sum(int l, int r) { return sum(l, r, 0, n, 1); }
    S sum(int l, int r, int tl, int tr, int i) {
        push(i);
        if (r <= tl || tr <= l) return e();
        if (l <= tl && tr <= r) return t[i];
```

```
        int tm = (tl + tr) / 2;
        return op(sum(l, r, tl, tm, i << 1), sum(l, r, tm, tr, i << 1 | 1));
    } // f3ba16
};
```

Sparse Table

Permite calcular soma de funções idempotentes em um intervalo de um array. Ex: Calcular o mínimo no intervalo $[L, R)$ Complexidade:

- Construção: $O(n \log(n))$.
- Consulta: $O(1)$.

Outros exemplos de funções incluem gcd, lcm e max.

175939 - dsa/sparse-table.cpp - 18 lines

```
template <class S, S (*op)(S, S)> struct SparseTable {
    vector<vector<S>> t;
    vi lg;
    void build(vector<S> &v) {
        int n = ssize(v);
        lg.assign(n + 1, 0);
        rep (i, 2, n + 1) lg[i] = lg[i >> 1] + 1;
        t.assign(lg[n] + 1, vector<S>(n));
        t[0] = v;
        rep (k, 1, lg[n])
            for (int i = 0; i + (1 << k) <= n; i++)
                t[k][i] = op(t[k - 1][i], t[k - 1][i + (1 << (k - 1))]);
    } // 986c70
    int query(int l, int r) {
        int k = lg[r - l];
        return op(t[k][l], t[k][r - (1 << k)]);
    } // f9bbfc
};
```

Disjoint Set Union

Representa uma lista de conjuntos, e suporta dois tipos de operações:

- $merge(X, Y)$: Junta os conjuntos dos números X e Y.
- $root(X)$: encontra o elemento representante do conjunto de X. Dois números estão no mesmo conjunto quando eles tem o mesmo representante.

49a155 - dsa/dsu.cpp - 17 lines

```
struct DSU {
    vi par, cnt;
    DSU(int n): par(n), cnt(n, 1) {
        iota(all(par), 0);
    }
    int root(int x) {
        if (par[x] == x) return x;
        return par[x] = root(par[x]);
    } // 809557
    bool merge(int x, int y) {
        x = root(x), y = root(y);
        if (x == y) return false;
        if (cnt[x] < cnt[y]) swap(x, y);
        cnt[x] += cnt[y], par[y] = x;
        return true;
    } // 3bef8c
};
```

Convolução

Calcula a convolução de dois arrays em $O(n \log(n))$. A convolução de dois arrays a e b é definida como um array c tal que:

$$c_k = \sum_{i+j=k} a_i b_j$$

FFT

bd675d - dsa/fft.cpp - 43 lines

```
using ld = double;
const long double PI = numbers::pi;
```

```
using VD = vector<ld>;
using C = complex<ld>;

void fft(vector<C> &a) {
    int n = a.size(), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1);
    for (static int k = 2; k < n; k <<= 1) {
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, PI / k);
        for (int i = k; i < 2 * k; i++)
            rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
    }
    VI rev(n);
    for (int i = 0; i < n; i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    for (int i = 0; i < n; i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
    for (int k = 1; k < n; k <<= 1)
        for (int i = 0; i < n; i += k << 1)
            for (int j = 0; j < k; j++) {
                auto x = (ld *)&rt[j + k], y = (ld *)&a[i + j + k];
                C z(x[0] * y[0] - x[1] * y[1], x[0] * y[1] + x[1] * y[0]);
                a[i + j + k] = a[i + j] - z;
                a[i + j] += z;
            }
} // ed7a4e

VD conv(VD &a, VD &b) {
    if (a.empty() || b.empty())
        return {};
    VD res(ssize(a) + ssize(b) - 1);
    int L = 32 - __builtin_clz(res.size()), n = 1 << L;
    vector<C> in(n), out(n);
    copy(a.begin(), a.end(), in.begin());
    for (int i = 0; i < b.size(); i++) in[i].imag(b[i]);
    fft(in);
    for (C &x : in) x *= x;
    for (int i = 0; i < n; i++) out[i] = in[-i & (n - 1)] - conj(in[i]);
    fft(out);
    for (int i = 0; i < res.size(); i++) res[i] = imag(out[i]) / (4 * n);
    return res;
} // fb4703
```

NTT

a359a7 - dsa/ntt.cpp - 53 lines

```
using VL = vector<i64>;

const int MOD = 998244353, root = 62;

i64 powm(i64 x, i64 e) {
    i64 r = 1;
    while (e) {
        if (e & 1) (r *= x) %= MOD;
        e >>= 1;
        (x *= x) %= MOD;
    }
    return r;
} // ae0a09

void ntt(VL &a) {
    int n = a.size(), k = bit_width(unsigned(n)) - 1;
    static array<i64, 1 << 23> r = {1, 1};
    for (static int t = 2, s = 2; t < n; t <<= 1, s++) {
        array z = {1ll, powm(root, MOD >> s)};
        for (int i = t; i < t << 1; i++) r[i] = r[i >> 1] * z[i & 1] % MOD;
    }
    static array<i64, 1 << 23> b, c;
    copy(a.begin(), a.end(), b.begin());
    for (int m = n; m >>= 1;) {
        for (int l = 0; l < n; l += m << 1)
            for (int i = 0; i < m; i++)
                c[l + i] = b[l + 2 * i], c[l + m + i] = b[l + 2 * i + 1];
        copy(c.begin(), c.begin() + n, b.begin());
    }
    for (int m = 1; m < n; m <<= 1) {
        for (int l = 0; l < n; l += m << 1)
            for (int i = 0; i < m; i++) {
                i64 z = r[m + i] * b[l + m + i] % MOD;
                c[l + i] = b[l + i] + z;
```

```
                c[l + m + i] = b[l + i] + (MOD - z);
            }
        for (int i = 0; i < n; i++) b[i] = c[i] < MOD ? c[i] : c[i] - MOD;
    }
    copy(b.begin(), b.begin() + n, a.begin());
} // 39c196

VL conv(VL &a, VL &b) {
    if (a.empty() || b.empty()) return {};
    int s = ssize(a) + ssize(b) - 1, B = 32 - __builtin_clz(s), n = 1 << B;
    int inv = powm(n, MOD - 2);
    VL L(a), R(b), out(n);
    L.resize(n), R.resize(n);
    ntt(L), ntt(R);
    for (int i = 0; i < n; ++i)
        out[-i & (n - 1)] = (i64)L[i] * R[i] % MOD * inv % MOD;
    ntt(out);
    return {out.begin(), out.begin() + s};
} // c28e2c
```

Max Add Segtree

Segtree que permite soma em range, e calcular o máximo em um range

c008ec - dsa/max-add-segtree.cpp - 41 lines

```
struct MaxAddSegtree {
    int n, h;
    vector<i64> t, d;
    MaxAddSegtree(int n) : n(n), h(32 - __builtin_clz(n)), t(2 * n), d(n) {}
    void apply(int p, i64 v) {
        t[p] += v;
        if (p < n) d[p] += v;
    } // 94936b
    void build(int p) {
        while (p > 1) p >>= 1, t[p] = max(t[p<<1], t[p<<1|1]) + d[p];
    } // c20980
    void push(int p) {
        for (int s = h; s > 0; --s) {
            int i = p >> s;
            if (d[i]) {
                apply(i<<1, d[i]);
                apply(i<<1|1, d[i]);
                d[i] = 0;
            }
        }
    } // 452305
    void update(int l, int r, i64 v) {
        l += n, r += n;
        int l0 = l, r0 = r;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) apply(l++, v);
            if (r & 1) apply(--r, v);
        }
        build(l0); build(r0 - 1);
    } // 8a7429
    i64 query(int l, int r) {
        l += n, r += n;
        push(l); push(r - 1);
        i64 res = -1e18;
        for (; l < r; l >>= 1, r >>= 1) {
            if (l & 1) res = max(res, t[l++]);
            if (r & 1) res = max(res, t[--r]);
        }
        return res;
    } // f655b5
};
```

Zeta e Mobius

Zeta: dada uma função $f : 2^{[1..n]} \to S$, encontra uma função $g : 2^{[1..n]} \to S$ tal que

$$g(A) = \sum_{B \subseteq A} f(B)$$

Mobius: realiza a transformação inversa.

eff0df - dsa/zeta.cpp - 28 lines

```
vi subsetZeta(vi a) {
```

```cpp
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep (i, 1, n)
            if (i & j) a[i] += a[i ^ j];
    return a;
} // b39c8d
vi subsetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i] -= a[i ^ j];
    return a;
} // 5ffc86
vi supersetZeta(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] += a[i];
    return a;
} // c5d556
vi supersetMobius(vi a) {
    int n = sz(a);
    for (int j = 1; j < n; j <<= 1)
        rep(i, 1, n)
            if (i & j) a[i ^ j] -= a[i];
    return a;
} // 128136
```

Árvore de Li Chao

Suporta dois tipos de operações:

- $insert(line)$: adiciona a reta $line(y = ax + b)$ ao conjunto de retas. Complexidade: $O(\log(N))$.
- $query(X)$: encontra a reta do conjunto que minimiza o valor $aX + b$. Complexidade: $O(\log(N))$.

Onde N é o tamanho do intervalo em que as retas atuam.

9e88be - dsa/li-chao.cpp - 29 lines

```cpp
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} // Replace with
    ↪ LLONG_MIN in case of finding max
    void insert(Line line) { insert(line, 0, n, 1); }
    void insert(Line line, int tl, int tr, int i) {
        int tm = tl + (tr - tl) / 2;
        if (line(tm) < t[i](tm)) swap(line, t[i]); // Replace with > in case
        ↪ of finding max
        if (tl + 1 == tr) return;
        if (t[i](tl) < line(tl)) insert(line, tm, tr, i << 1 | 1); // Replace
        ↪ with > in case of finding max
        else insert(line, tl, tm, i << 1);
    } // f784a4
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN in case of
        ↪ finding max
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with > in case of
        ↪ finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Árvore de Li Chao++

Suporta dois tipos de operações:

- $insert(line, L, R)$: adiciona a reta $line(y = ax+b)$ ao conjunto de retas,

atuando no intervalo L, R. Complexidade: $O(\log^2(R-L))$.

- $query(X)$: encontra a reta do conjunto que minimiza o valor $aX + b$. Complexidade: $O(\log(L))$.

8bf7d9 - dsa/li-chao-extended.cpp - 36 lines

```
struct Line {
    ll a, b;
    ll operator()(int x) { return a * x + b; }
};
struct LiChao {
    vector<Line> t;
    int n;
    LiChao(int N): t(4 * N, {0, LLONG_MAX}), n(N) {} // Replace with
    ↪  LLONG_MIN in case of finding max
    void insert(Line line, int L, int R) { insert(line, L, R, 0, n, 1); }
    void insert(Line line, int L, int R, int tl, int tr, int i) {
        int tm = tl + (tr - tl) / 2;
        if (L >= tr || R <= tl) return;
        if (L <= tl && tr <= R) {
            if (line(tm) < t[i](tm)) swap(line, t[i]); // Replace with > in
            ↪  case of finding max
            if (tl + 1 == tr) return;
            if (t[i](tl) < line(tl)) insert(line, L, R, tm, tr, i << 1 | 1);
            ↪  // Replace with > in case of finding max
            else insert(line, L, R, tl, tm, 1 << 1);
        } else {
            if (tl + 1 == tr) return;
            insert(line, L, R, tm, tr, i << 1 | 1);
            insert(line, L, R, tl, tm, i << 1);
        }
    } // 0b7e08
    Line query(int x) {
        Line ans = {0, LLONG_MAX}; // Replace with LLONG_MIN in case of
        ↪  finding max
        query(x, ans, 0, n, 1);
        return ans;
    } // 2aca6b
    void query(int x, Line &line, int tl, int tr, int i) {
        if (t[i](x) < line(x)) line = t[i]; // Replace with > in case of
        ↪  finding max
        int tm = tl + (tr - tl) / 2;
        if (tl + 1 == tr) return;
        if (x < tm) query(x, line, tl, tm, i << 1);
        else query(x, line, tm, tr, i << 1 | 1);
    } // 973d96
};
```

Árvore Cartesiana

Constrói uma árvore cartesiana dos inteiros $0, 1, \ldots, n-1$ a partir de um critério de comparação. A árvore é retornada no formato de um array par de tal forma que par_i é o pai do nó i na árvore. A raiz da árvore é o nó mínimo de acordo com o critério de comparação. Complexidade: $O(n)$.

0b53ed - dsa/cartesian-tree.cpp - 20 lines

```
vi cartTree(int n, auto le) {
    int last;
    vi st, par(n);
    rep (i, 0, n) {
        par[i] = i, last = -1;
        while (sz(st) && le(par[i], st.back())) {
            if (last != -1) par[last] = st.back();
            last = st.back();
            st.pop_back();
        }
        if (last != -1) par[last] = i;
        st.push_back(i);
    }
    while (sz(st) > 1) {
        last = st.back();
        st.pop_back();
        par[last] = st.back();
    }
    return par;
} // 0b53ed
```

Árvore de Busca Binária Aleatória

6a6cd3 - dsa/rbst.cpp - 65 lines

```
static mt19937 rng;

template<class T>
struct RBST {
    struct Node {
        T v;
        int cnt = 1;
        Node *ch[2] = {0, 0};
        ~Node() {
            for (int i: {0, 1}) if (ch[i]) delete ch[i];
        }
    };
    static Node * update(Node *n) {
        if (!n) return 0;
        n->cnt = 1;
        for (int i: {0, 1}) n->cnt += n->ch[i] ? n->ch[i]->cnt : 0;
        return n;
    } // 0fee8d
    static Node *merge(Node *l, Node *r) {
        if (!l || !r) return l ? l : r;
        if (rng() % (l->cnt + r->cnt) < l->cnt) {
            l->ch[1] = merge(l->ch[1], r);
            return update(l);
        } else {
            r->ch[0] = merge(l, r->ch[0]);
            return update(r);
        }
    } // f0d9c8
    static void split(Node *rt, auto cond, Node *&l, Node *&r) {
        if (!rt) return;
        if (cond(rt->v)) {
            r = rt, rt = r->ch[0], r->ch[0] = 0;
            split(rt, cond, l, r->ch[0]);
            update(r);
        } else {
            l = rt, rt = l->ch[1], l->ch[1] = 0;
            split(rt, cond, l->ch[1], r);
            update(l);
        }
    } // 44f1a1
    static void splitAt(Node *rt, int i, Node *&l, Node *&r) {
        if (!rt) return;
        int k = rt->ch[0] ? rt->ch[0]->cnt : 0;
        if (i <= k) {
            r = rt, rt = r->ch[0], r->ch[0] = 0;
            splitAt(rt, i, l, r->ch[0]);
            update(r);
            return;
        }
        i -= k + 1;
        l = rt, rt = l->ch[1], l->ch[1] = 0;
        splitAt(rt, i, l->ch[1], r);
        update(l);
    } // a19d8b
    static Node *front(Node * rt) {
        if (!rt || !rt->ch[0]) return rt;
        return front(rt->ch[0]);
    } // b1a234
    static Node *back(Node *rt) {
        if (!rt || !rt->ch[1]) return rt;
        return back(rt->ch[1]);
    } // 441301
    Node *rt = 0;
    RBST() : rt(0) {}
};
```

Treap implícita

Treap implícita para sequência: suporta `split`/`merge` por posição, inserção, remoção, reversão lazy e soma em intervalo.

ad716f - dsa/treap.cpp - 69 lines

```
struct Node;
using TN = Node*;

struct Node {
    int y = rng(), cnt, v;
    ll sum;
```

```
    bool rev = 0;
    TN l = 0, r = 0;
    Node(int v) : cnt(1), v(v), sum(v) {}
};

int cnt(TN t) { return t ? t->cnt : 0; }
ll sum(TN t) { return t ? t->sum : 0; }

void apply_rev(TN t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
} // 907c51
void push(TN t) {
    if (!t || !t->rev) return;
    apply_rev(t->l);
    apply_rev(t->r);
    t->rev = 0;
} // b46a2a
void pull(TN t) {
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->v + sum(t->l) + sum(t->r);
} // 6ee5eb

void split(TN t, int k, TN& a, TN& b) {
    if (!t) return a = b = 0, void();
    push(t);
    if (cnt(t->l) >= k) split(t->l, k, a, t->l), b = t;
    else split(t->r, k - cnt(t->l) - 1, t->r, b), a = t;
    pull(t);
} // b34b6d
TN merge(TN a, TN b) {
    if (!a || !b) return a ? a : b;
    if (a->y > b->y) return push(a), a->r = merge(a->r, b), pull(a), a;
    return push(b), b->l = merge(a, b->l), pull(b), b;
} // 529963

void ins(TN& t, int p, int v) {
    TN a, b;
    split(t, p, a, b);
    t = merge(merge(a, new Node(v)), b);
} // c2b578
void era(TN& t, int p) {
    TN a, b, c;
    split(t, p, a, b);
    split(b, 1, b, c);
    delete b;
    t = merge(a, c);
} // cf32a5
TN kth(TN t, int p) {
    while (t) {
        push(t);
        if (cnt(t->l) == p) return t;
        if (cnt(t->l) > p) t = t->l;
        else p -= cnt(t->l) + 1, t = t->r;
    }
    return 0;
} // 4ba488

void sep(TN t, int l, int r, TN& a, TN& b, TN& c) {
    split(t, r, b, c);
    split(b, l, a, b);
} // 4fa0ee
```

Treap de conjunto ordenado

Treap por chave para conjunto ordenado: inserção, remoção, busca, k-ésimo elemento, contagem $\leq x$, predecessor e sucessor.

6ed571 - dsa/treap-set.cpp - 81 lines

```
struct TS {
    i64 v;
    i32 pr, sz;
    TS *l, *r;
    TS(i64 V) : v(V), pr((rand()<<16)^rand()), sz(1), l(0), r(0) {}
};

int sz(TS* t) { return t ? t->sz : 0; }
void pull(TS* t) { if (t) t->sz = 1 + sz(t->l) + sz(t->r); }
```

```
void split(TS* t, int key, TS*& l, TS*& r) {
    if (!t) l = r = 0;
    else if (t->v <= key) split(t->r, key, t->r, r), l = t;
    else split(t->l, key, l, t->l), r = t;
    pull(t);
} // 0240c2

TS* merge(TS* l, TS* r) {
    if (!l || !r) return l ? l : r;
    if (l->pr > r->pr) {
        l->r = merge(l->r, r);
        pull(l);
        return l;
    } else {
        r->l = merge(l, r->l);
        pull(r);
        return r;
    }
} // 244be4

bool find(TS* t, int v) {
    while (t) {
        if (t->v == v) return 1;
        t = v < t->v ? t->l : t->r;
    }
    return 0;
} // 3b0421

void insert(TS*& t, TS* it) {
    if (!t) t = it;
    else if (it->pr > t->pr) split(t, it->v, it->l, it->r), t = it;
    else insert(it->v < t->v ? t->l : t->r, it);
    pull(t);
} // 75b7ab

void erase(TS*& t, int v) {
    if (!t) return;
    if (t->v == v) t = merge(t->l, t->r);
    else erase(v < t->v ? t->l : t->r, v);
    pull(t);
} // 1f8476

int kth(TS* t, int k) { // 1-indexed, assumes 1 <= k <= sz(t)
    if (sz(t->l) + 1 == k) return t->v;
    if (k <= sz(t->l)) return kth(t->l, k);
    return kth(t->r, k - sz(t->l) - 1);
} // 41972c

int cntLE(TS* t, i64 v) {
    if (!t) return 0;
    if (t->v <= v) return sz(t->l) + 1 + cntLE(t->r, v);
    return cntLE(t->l, v);
} // 8e7469

TS* pred(TS* t, i64 v) { // max key <= x
    TS* ans = 0;
    while (t) {
        if (t->v <= v) ans = t, t = t->r;
        else t = t->l;
    }
    return ans;
} // 9dda3d

TS* succ(TS* t, i64 v) { // min key >= x
    TS* ans = 0;
    while (t) {
        if (t->v >= v) ans = t, t = t->l;
        else t = t->r;
    }
    return ans;
} // dcf6b7
```

Grafos

Caminho Euleriano

Encontra um caminho euleriano em um grafo, se existir. Recebe uma lista de adjacência da forma $[x, e]$, onde x é o vizinho e e é o id da aresta. Para grafos não direcionados, basta usar o mesmo id para arestas indo e voltando.

Complexidade: $O(V + E)$.

927481 - graph/eulerian-walk.cpp - 23 lines

```
pair<vi, vi> eulerWalk(vector<vector<pii>>& g, int m, int s = 0) {
    int n = sz(g);
    vi path, walk, d(n), vis(m), it(n);
    vector<pii> st = {{s, -1}};
    d[s]++; // remove to force cycles
    while (sz(st)) {
        auto [v, e] = st.back();
        if (it[v] == sz(g[v])) {
            path.pb(v), walk.pb(e);
            st.pop_back();
        } else {
            auto [u, id] = g[v][it[v]++];
            if (!vis[id]) {
                d[v]--, d[u]++;
                vis[id] = 1;
                st.pb({u, id});
            }
        }
    }
    for (int x : d) if (x < 0) return {{}, {}};
    if (sz(walk) != m + 1) return {{}, {}};
    return { vi(path.rbegin(), path.rend()), vi(walk.rbegin() + 1,
    ↪  walk.rend()) };
} // 927481
```

Encontrando qualquer caminho euleriano

Em grafos direcionados:

- Se exatamente um vértice tem $d_{out} - d_{in} = 1$, então comece nesse vértice.
- Se todos os vértices tem $d_{out} = d_{in}$, pode começar em qualquer vértice com $d_{out} > 0$.
- Caso contrário, não existe caminho euleriano.

Em grafos não direcionados:

- Se exatamente dois vértices tem deg ímpar, então comece em qualquer um deles.
- Se todos os vértices tem deg par, pode começar em qualquer vértice com $deg > 0$.
- Caso contrário, não existe caminho euleriano.

Max Flow

Dinic

Encontra o fluxo máximo de s a t dado um grafo com arestas e capacidades. Complexidade: $O(N^2M)$.

- $addEdge(from, to, capacity)$: adiciona uma aresta ao grafo de $from$ pra to com capacidade $capacity$.
- $flow(s, t)$: encontra o fluxo máximo de s para t.
- $flow(s, t, max)$: o mesmo que a última, mas limita o fluxo a max.

Nota: A complexidade também é limitada pelo valor do fluxo máximo, se o fluxo máximo é F, então a complexidade é menor ou igual a $O(FM)$.

11f7ad - graph/dinic.cpp - 55 lines

```
struct Dinic {
    struct Edge {
        ll s, t, f, c, r;
    };
    vector<Edge> e;
    vector<vi> adj;
    vi ne, lvl;
    Dinic(int n): adj(n), ne(n), lvl(n) {}
    void addEdge(int a, int b, ll c) {
        adj[a].push_back(sz(e));
        e.emplace_back(a, b, 0, c, sz(e) + 1);
        adj[b].push_back(sz(e));
        e.emplace_back(b, a, 0, 0, sz(e) - 2);
    } // 381300
    bool bfs(int s, int t) {
        queue<int> q;
        q.push(s);
        fill(all(lvl), -1);
        lvl[s] = 0;
        while (sz(q)) {
```

```
            int x = q.front();
            q.pop();
            for (int i: adj[x]) {
                if (e[i].f == e[i].c) continue;
                if (lvl[e[i].t] != -1) continue;
                lvl[e[i].t] = lvl[x] + 1;
                q.push(e[i].t);
            }
        }
        return lvl[t] == -1;
    } // 1625d2
    ll dfs(int x, int t, ll f) {
        if (f == 0) return 0;
        if (x == t) return f;
        for (int &te = ne[x]; te < sz(adj[x]); te++) {
            int i = adj[x][te], y = e[i].t;
            if (lvl[y] != lvl[x] + 1) continue;
            if (ll df = dfs(y, t, min(f, e[i].c - e[i].f))) {
                e[i].f += df;
                e[i ^ 1].f -= df;
                return df;
            }
        }
        return 0;
    } // d7b05b
    ll flow(int s, int t, ll mx = LLONG_MAX) {
        ll mf = 0;
        while (true) {
            if (bfs(s, t)) break;
            fill(all(ne), 0);
            while (ll f = dfs(s, t, mx - mf)) mf += f;
        }
        return mf;
    } // 53644f
};
```

Min Cost Flow

MCMF (SPFA)

Encontra o fluxo de custo mínimo (e fluxo máximo) de s a t. Suporta arestas com custo negativo.

- $addEdge(from, to, cap, cost)$: adiciona aresta com capacidade cap e custo $cost$ por unidade de fluxo.
- $flow(s, t)$: retorna $\{max_flow, min_cost\}$.

Complexidade: $O(VE \cdot flow)$ no pior caso.

7114ac - graph/mcf.cpp - 43 lines

```
struct MCMF {
    struct Edge { i32 s, t; i64 f, c, w; };
    vector<Edge> e;
    VVI adj;
    vector<i64> dist;
    VI par;
    vector<bool> inq;
    MCMF(int n): adj(n), dist(n), par(n), inq(n) {}
    void addEdge(int a, int b, i64 c, i64 w) {
        adj[a].push_back(e.size());
        e.push_back({a, b, 0, c, w});
        adj[b].push_back(e.size());
        e.push_back({b, a, 0, 0, -w});
    } // e5d2d3
    bool spfa(int s, int t) {
        fill(dist.begin(), dist.end(), LLONG_MAX);
        fill(inq.begin(), inq.end(), false);
        queue<int> q;
        dist[s] = 0; inq[s] = true; q.push(s);
        while (!q.empty()) {
            int x = q.front(); q.pop(); inq[x] = false;
            for (int i: adj[x]) {
                auto &ei = e[i];
                if (ei.f == ei.c || dist[x] + ei.w >= dist[ei.t]) continue;
                dist[ei.t] = dist[x] + ei.w;
                par[ei.t] = i;
                if (!inq[ei.t]) { inq[ei.t] = true; q.push(ei.t); }
            }
```

```
        }
        return dist[t] != LLONG_MAX;
    } // fbb602
    // returns {max_flow, min_cost}
    pair<i64, i64> flow(int s, int t) {
        i64 mf = 0, mc = 0;
        while (spfa(s, t)) {
            i64 f = LLONG_MAX;
            for (int x = t; x != s; ) { int i = par[x]; f = min(f, e[i].c -
            ↪  e[i].f); x = e[i].s; }
            for (int x = t; x != s; ) { int i = par[x]; e[i].f += f; e[i^1].f
            ↪  -= f; x = e[i].s; }
            mf += f; mc += f * dist[t];
        }
        return {mf, mc};
    } // c2e4d0
};
```

Árvores

Base

Inclui todas as funcionalidades básicas de uma árvore para serem usadas no resto dos templates de árvore. Depois de adicionar todas as arestas, use $build(root)$ para construir a árvore com raiz em $root$.

00b4f5 - tree/base.cpp - 25 lines

```
struct Tree {
    VVI adj;
    VI cnt, par, dst, tin, tout, nxt;
    int t;
    Tree(int n): adj(n), cnt(n), par(n), dst(n), tin(n), tout(n), nxt(n),
    ↪  t(0) {}
    void addEdge(int a, int b) { adj[a].push_back(b); adj[b].push_back(a); }
    void build(int r = 0) { build(r, -1); par[r] = r; nxt[r] = r; prepare(r);
    ↪  }
    void build(int x, int pre) {
        cnt[x] = 1;
        for (int &y: adj[x]) if (y != pre) {
            par[y] = x, dst[y] = dst[x] + 1;
            build(y, x);
            cnt[x] += cnt[y];
            if (cnt[y] > cnt[adj[x][0]]) swap(adj[x][0], y);
        }
    } // 6e5d12
    void prepare(int x) {
        tin[x] = t++;
        for (int y: adj[x]) if (y != par[x]) {
            nxt[y] = y == adj[x][0] ? nxt[x] : y;
            prepare(y);
        }
        tout[x] = t;
    } // 1b3e55
};
```

Movimento em árvores

Inclui funcionalidades de movimento em árvores:

- $lca(x, y)$ encontra o menor ancestral comum entre x e y. Complexidade: $O(\log(n))$.
- $up(x, d)$ encontra o d-ésimo ancestral de x. Complexidade: $O(\log(n))$.
- $dist(x, y)$ encontra a distância entre os vértices x e y (em número de arestas no caminho). Complexidade: $O(\log(n))$.
- $move(x, y, d)$ encontra o d-ésimo vértice no menor caminho de x a y. Complexidade: $O(\log(n))$.

53a574 - tree/tree-move.cpp - 32 lines

```
struct TreeMove {
    int n;
    VI w, tm;
    Tree &t;
    TreeMove(Tree &t): n(t.adj.size()), w(n), tm(n), t(t) {
        for (int i = 0; i < n; i++) {
            w[t.tin[i]] = i;
            tm[i] = t.tin[i];
```

```
        }
    }
    int lca(int x, int y) {
        while (t.nxt[x] != t.nxt[y]) {
            if (t.dst[t.nxt[x]] > t.dst[t.nxt[y]]) x = t.par[t.nxt[x]];
            else y = t.par[t.nxt[y]];
        }
        return w[min(tm[x], tm[y])];
    } // 74d125
    int up(int x, int d) {
        while (t.dst[x] - t.dst[t.nxt[x]] < d) {
            d -= t.dst[x] - t.dst[t.nxt[x]] + 1;
            x = t.par[t.nxt[x]];
        }
        return w[tm[x] - d];
    } // 3fb308
    int move(int x, int y, int d) {
        int l = lca(x, y), ld = t.dst[l], dx = t.dst[x] - ld, dy = t.dst[y] -
        ↪ ld;
        return d > dx ? up(y, dx + dy - d) : up(x, d);
    } // ea0e64
    int dist(int x, int y) {
        return t.dst[x] + t.dst[y] - 2 * t.dst[lca(x, y)];
    } // 0a8d1a
};
```

HLD

Obtém a decomposição do caminho de x a y em partes do array de dfs. É garantido que vão existir no máximo $O(\log(n))$ partes. O argumento *ord* serve para indicar que é preciso diferenciar caminhos subindo e descendo (útil quando a ordem dos vértices no caminho importa).

ac6fba - tree/hld-path.cpp - 19 lines

```
VII hldPath(Tree &t, int x, int y, bool ord = false) {
    VII p, rp;
    while (t.nxt[x] != t.nxt[y]) {
        if (t.dst[t.nxt[x]] < t.dst[t.nxt[y]]) {
            rp.emplace_back(t.tin[t.nxt[y]], t.tin[y]);
            y = t.par[t.nxt[y]];
        } else {
            if (ord) p.emplace_back(t.tin[x], t.tin[t.nxt[x]]);
            else p.emplace_back(t.tin[t.nxt[x]], t.tin[x]);
            x = t.par[t.nxt[x]];
        }
    }
    if (t.dst[x] <= t.dst[y] || ord)
        p.emplace_back(t.tin[x], t.tin[y]);
    else p.emplace_back(t.tin[y], t.tin[x]);
    reverse(rp.begin(), rp.end());
    p.insert(p.begin(), rp.begin(), rp.end());
    return p;
} // ac6fba
```

LCA

Permite calcular o LCA de dois vértices em $O(1)$.

2539ab - tree/lca.cpp - 23 lines

```
using RMQ = SparseTable<int, [](int a, int b) { return min(a, b); }>;

struct LCA {
    int t = 0;
    VI tm, path, ret;
    RMQ rmq;
    LCA(VVI &adj, int r = 0) : tm(adj.size()) {
        dfs(adj, r, -1);
        rmq.build(ret);
    }
    void dfs(VVI &adj, int x, int pre) {
        tm[x] = t++;
        for (int y: adj[x]) if (y != pre) {
            path.push_back(x), ret.push_back(tm[x]);
            dfs(adj, y, x);
        }
    } // c5a5ad
    int lca(int x, int y) {
        if (x == y) return x;
```

```
        tie(x, y) = minmax(tm[x], tm[y]);
        return path[rmq.query(x, y)];
    } // e8105a
};
```

Tree Matching

Emparelhamento máximo em uma árvore (0-indexado). Usa DFS guloso em pós-ordem: emparelha um filho com o pai quando ambos estão livres. Preenche $match[v]$ com o parceiro de v, ou -1 se não emparelhado. Complexidade: $O(n)$.

7957a6 - tree/tree-matching.cpp - 14 lines

```
// Maximum matching on tree (0-indexed adjacency list)
// Returns matching size; match[v] = matched partner or -1
int treeMatch(VVI &adj, VI &match, int root = 0) {
    int n = adj.size(), ans = 0;
    match.assign(n, -1);
    function<bool(int, int)> dfs = [&](int v, int p) -> bool {
        for (int u: adj[v]) if (u != p && dfs(u, v) && match[v] == -1) {
            match[v] = u; match[u] = v; ans++;
        }
        return match[v] == -1;
    };
    dfs(root, -1);
    return ans;
} // 7957a6
```

Strings

Suffix Array

Dada uma string S, ordena todos os sufixos em ordem lexicográfica. Retorna um array SA tal que

$$SA_i = \text{índice do começo do } i\text{-ésimo sufixo em ordem lexicográfica}$$

Complexidade: $O(n \log(n))$, onde n é o tamanho da string.

7de5e7 - strings/suffix-array.cpp - 28 lines

```
vi sufArr(string &s) { // Could also be vector<T> &s
    int n = sz(s);
    vi c(n), d(n), e(n), sb(n), sa(n), cnt(n + 1);
    iota(all(sa), 0);
    sort(all(sa), [&](int i, int j) { return s[i] < s[j]; });
    c[sa[0]] = 1;
    for (int i = 1; i < n; i++)
        c[sa[i]] = c[sa[i - 1]] + (s[sa[i]] != s[sa[i - 1]]);
    for (int k = 1; c[sa[n - 1]] != n; k <<= 1) {
        rep (i, 0, n) d[i] = i + k < n ? c[i + k] : 0;
        auto srt = [&](auto &C) {
            fill(all(cnt), 0);
            rep (i, 0, n) cnt[C[i] + 1]++;
            rep (i, 0, n) cnt[i + 1] += cnt[i];
            for (int i: sa) sb[cnt[C[i]]++] = i;
            swap(sa, sb);
        };
        srt(d); srt(c);
        int lc = 0, ld = 0, le = 0;
        for (int i: sa) {
            if (c[i] != lc || d[i] != ld) e[i] = le + 1;
            else e[i] = le;
            lc = c[i], ld = d[i], le = e[i];
        }
        swap(c, e);
    }
    return sa;
} // 7de5e7
```

Maior prefixo comum (LCP)

Dada uma string e o suffix array dela, calcula para cada duas strings consecutivas no suffix array, qual o maior prefixo comum delas. Mais formalmente,

calcula um array LCP tal que

$$LCP_i = \text{maior prefixo comum dos sufixos } SA_i \text{ e } SA_{i+1}$$

Complexidade: $O(n)$. Onde n é o tamanho da string.

69e59c - strings/lcp.cpp - 16 lines

```
vi lcpArr(string &s, vi &sa) {
    int n = sz(s), k = 0;
    vi r(n), lcp(n);
    rep (i, 0, n) r[sa[i]] = i;
    rep (i, 0, n) {
        if (r[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = sa[r[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
        lcp[r[i]] = k;
        if (k) k--;
    }
    return lcp;
} // 69e59c
```

KMP

Dada uma string S, calcula um array pre de forma que

$$pre_i = \text{ maior valor de } k < i \text{ tal que } S_0S_1\cdots S_{k-1} = S_{i-k+1}S_{i-k+2}\cdots S_i$$

Complexidade: $O(n)$.

cb52b3 - strings/kmp.cpp - 10 lines

```
vi kmp(string s) {
    int n = sz(s), len = 0;
    vi pre(n);
    rep (i, 1, n) {
        while (len > 0 && s[i] != s[len]) len = pre[len - 1];
        if (s[i] == s[len]) len++;
        pre[i] = len;
    }
    return pre;
} // cb52b3
```

Geometria

Base

Base para usar em outros templates de geometria.

440b05 - geometry/prelude.cpp - 9 lines

```
using D = i64;
using point = complex<D>;

D dot(point a, point b) {
    return real(conj(a) * b);
} // c6b21e
D cross(point a, point b) {
    return imag(conj(a) * b);
} // a8ce1d
```

Interseção de retas

Encontra o ponto de interseção de duas retas.

cc6127 - geometry/line-intersection.cpp - 5 lines

```
// D = double;

point lineIntersect(point a1, point d1, point a2, point d2) {
    return a1 + cross(a2 - a1, d2) / cross(d1, d2) * d1;
} // cc6127
```

Interseção de segmentos

Verifica se dois segmentos de reta se intersectam.

3b079c - geometry/segment-intersection.cpp - 10 lines

```
#define CK(a, b, t) max(t(a[0]), t(a[1])) < min(t(b[0]), t(b[1]))
#define CKK(a, b, t) (s(a[0], a[1], b[t]) == 0 && dot(b[t] - a[0], b[t] -
↪  a[1]) <= 0)

bool segInterChk(point a, point b, point c, point d) {
    array p = {a, b}, q = {c, d};
    auto s = [](point p, point q, point r) { return cross(q - p, r - p); };
    if (CK(p, q, real) || CK(p, q, imag) || CK(q, p, real) || CK(q, p, imag))
    ↪  return false;
    if (CKK(p, q, 0) || CKK(p, q, 1) || CKK(q, p, 0) || CKK(q, p, 1)) return
    ↪  true;
    return (s(a, b, c) > 0) != (s(a, b, d) > 0) && (s(c, d, a) > 0) != (s(c,
    ↪  d, b) > 0);
} // c22bf9
```

Área de um polígono

Calcula a área de um polígono dada a lista de vértices. A área é positiva se os vértices estão no sentido anti-horário, e negativa caso contrário. Complexidade: $O(n)$.

aa075b - geometry/polygon-area.cpp - 6 lines

```
D polyArea(vector<point> &pt) {
    int n = pt.size();
    D s = 0;
    for (int i = 2; i < n; i++) s += cross(pt[i - 1] - pt[0], pt[i] - pt[0]);
    return s;
} // aa075b
```

Verificar se um ponto está dentro de um polígono

Verifica se um ponto está dentro de um polígono convexo. Use $border = \text{false}$ para não incluir pontos na borda do polígono. Complexidade: $O(\log(n))$.

b44cc8 - geometry/point-in-polygon.cpp - 10 lines

```
bool pointInPoly(vector<point> &pt, point q, bool border = true) {
    int n = pt.size(), t = border, l = 1, r = n - 1;
    auto v = [&](int i) { return cross(q - pt[0], pt[i] - pt[0]); };
    if (v(l) >= t || v(r) <= -t) return 0;
    while (r - l > 1) {
        int m = (l + r) / 2;
        (v(m) > 0 ? r : l) = m;
    }
    return cross(pt[r] - pt[l], pt[l] - q) < t;
} // b44cc8
```

Convex hull

Encontra o convex hull de um conjunto de pontos. O resultado é uma lista de índices dos pontos do convex hull no sentido anti-horário. Complexidade: $O(n \log(n))$.

35ea37 - geometry/convex-hull.cpp - 24 lines

```
vector<int> convHull(vector<point> &pt) {
    int n = sz(pt), m;
    vi h, ord(n);
    auto add = [&]() {
        vi st;
        for (int i: ord) {
            while ((m = sz(st)) > 1) {
                point a = pt[st[m - 1]], b = pt[st[m - 2]], c = pt[i];
                if (cross(b - a, c - a) < 0) break; // > for clockwise, <= to
                ↪  include non-vertices
                st.pop_back();
            }
            st.push_back(i);
        }
```

```
        st.pop_back();
        h.insert(h.end(), all(st));
    };
    iota(all(ord), 0);
    auto top = [](auto a) { return make_pair(real(a), imag(a)); };
    sort(all(ord), [&](int i, int j) { return top(pt[i]) > top(pt[j]); });
    add();
    reverse(all(ord));
    add();
    return h;
} // 35ea37
```

Menor distância euclideana

Encontra a menor distância euclideana entre qualquer par de pontos. Requer $D = double$. Complexidade: $O(n \log n)$.

76d39a - geometry/closest-pair.cpp - 18 lines

```
// D = double;
// Minimum euclidean distance between any two points. O(n log n).
D closestPair(vector<point> pts) {
    sort(pts.begin(), pts.end(), [](point a, point b) {
        return make_pair(real(a), imag(a)) < make_pair(real(b), imag(b));
    });
    D best = abs(pts[0] - pts[1]);
    set<pair<D, D>> S;
    int l = 0;
    for (auto p: pts) {
        D x = real(p), y = imag(p);
        while (x - real(pts[l]) > best) S.erase({imag(pts[l]),
        ↪  real(pts[l++])});
        for (auto it = S.lower_bound({y - best, -1e18}); it != S.end() &&
        ↪  it->first <= y + best; it++)
            best = min(best, abs(p - point(it->second, it->first)));
        S.insert({y, x});
    }
    return best;
} // 76d39a
```

Interseção de círculo e reta

Encontra os pontos de interseção entre um círculo (centro c, raio r) e uma reta (ponto a, direção d). Retorna 0, 1 ou 2 pontos. Requer $D = double$.

b20456 - geometry/circle-line.cpp - 11 lines

```
// D = double;
// Circle center c, radius r. Line through a in direction d.
// Returns 0, 1 or 2 intersection points.
vector<point> circLineInter(point c, D r, point a, point d) {
    point f = a + dot(c - a, d) / dot(d, d) * d;
    D h2 = r * r - norm(c - f);
    if (h2 < -1e-9) return {};
    D dt = sqrt(max((D)0, h2)) / abs(d);
    if (h2 < 1e-9) return {f};
    return {f - dt * d, f + dt * d};
} // b20456
```

Interseção de círculos

Encontra os pontos de interseção entre dois círculos (c_1, r_1) e (c_2, r_2). Retorna 0, 1 ou 2 pontos. Requer $D = double$.

29278b - geometry/circle-circle.cpp - 14 lines

```
// D = double;
// Returns 0, 1 or 2 intersection points of circles (c1,r1) and (c2,r2).
vector<point> circCircInter(point c1, D r1, point c2, D r2) {
    point d = c2 - c1;
    D dist = abs(d);
    if (dist < 1e-9) return {};
    D a = (norm(d) + r1 * r1 - r2 * r2) / (2 * dist);
    D h2 = r1 * r1 - a * a;
    if (h2 < -1e-9) return {};
    point mid = c1 + a / dist * d;
    if (h2 < 1e-9) return {mid};
    point perp = point(-imag(d), real(d)) * (sqrt(max((D)0, h2)) / dist);
    return {mid - perp, mid + perp};
```

```
} // 29278b
```

Tangentes comuns a dois círculos

Retorna pares $\{p_1, p_2\}$ de pontos de tangência (um em cada círculo), com até 4 tangentes. $s = 1$: tangentes externas; $s = -1$: tangentes internas. Requer $D = double$.

f72e1a - geometry/circle-tangents.cpp - 18 lines

```
// D = double;
// Common tangents of circles (c1,r1) and (c2,r2).
// Returns tangent contact pairs {p on c1, p on c2}, up to 4.
// s=1: external tangents (circles same side); s=-1: internal (opposite
↪  sides).
vector<pair<point, point>> circTangents(point c1, D r1, point c2, D r2) {
    vector<pair<point, point>> res;
    for (int s: {1, -1}) {
        D r = s * r2, dr = r1 - r;
        point d = c2 - c1;
        D l2 = norm(d), h2 = max((D)0, l2 - dr * dr);
        if (l2 < dr * dr - 1e-9) continue;
        for (int t: {1, -1}) {
            point n = (dr * d + point(-imag(d), real(d)) * (t * sqrt(h2))) /
            ↪  l2;
            res.push_back({c1 + r1 * n, c2 + r * n});
        }
    }
    return res;
} // f72e1a
```

Soma Minkowski de polígonos convexos

Dado dois polígonos convexos a e b em sentido anti-horário, retorna a soma de Minkowski $a \oplus b$ também em sentido anti-horário. Complexidade: $O(|a|+|b|)$.

28b8cd - geometry/minkowski.cpp - 19 lines

```
// Minkowski sum of convex polygons a and b (both CCW).
// Result is also CCW.
vector<point> minkowski(vector<point> a, vector<point> b) {
    auto cmp = [](point p, point q) {
        return make_pair(imag(p), real(p)) < make_pair(imag(q), real(q));
    };
    rotate(a.begin(), min_element(a.begin(), a.end(), cmp), a.end());
    rotate(b.begin(), min_element(b.begin(), b.end(), cmp), b.end());
    int n = a.size(), m = b.size();
    a.push_back(a[0]); a.push_back(a[1]);
    b.push_back(b[0]); b.push_back(b[1]);
    vector<point> res;
    for (int i = 0, j = 0; i < n || j < m; ) {
        res.push_back(a[i] + b[j]);
        D c = cross(a[i+1] - a[i], b[j+1] - b[j]);
        i += (c >= 0); j += (c <= 0);
    }
    return res;
} // 28b8cd
```

Convex hull online

Mantém o convex hull dinamicamente à medida que pontos são inseridos. $add(p)$: insere ponto. $has(p)$: verifica se p está dentro do hull. Requer $D = double$. Complexidade: $O(\log n)$ por operação.

8c0cac - geometry/convex-hull-online.cpp - 33 lines

```
// D = double;
// Online convex hull: insert points one by one, query containment.
// add(p): insert point p. has(p): true if p is inside or on the hull.
struct ConvHullDyn {
    using M = map<D, D>;
    M up, lo;
    bool bad(M &h, M::iterator y) {
        if (y == h.begin() || next(y) == h.end()) return false;
        auto a = prev(y), b = next(y);
        return (b->first - a->first) * (y->second - a->second) <=
               (b->second - a->second) * (y->first - a->first);
    } // ef404b
```

```
    void ins(M &h, D x, D y) {
        if (h.count(x) && h[x] >= y) return;
        h[x] = y;
        auto it = h.find(x);
        if (bad(h, it)) { h.erase(it); return; }
        while (it != h.begin() && bad(h, prev(it))) h.erase(prev(it));
        while (next(it) != h.end() && bad(h, next(it))) h.erase(next(it));
    } // d3fda9
    void add(point p) { ins(up, real(p), imag(p)); ins(lo, real(p),
    ↪  -imag(p)); }
    bool has(point p) {
        if (up.size() < 2) return false;
        auto it = up.lower_bound(real(p));
        auto jt = lo.lower_bound(real(p));
        if (it == up.end() || it == up.begin()) return false;
        if (jt == lo.end() || jt == lo.begin()) return false;
        auto bU = it, aU = prev(it), bL = jt, aL = prev(jt);
        D yu = aU->second + (bU->second - aU->second) * (real(p) - aU->first)
        ↪  / (bU->first - aU->first);
        D yl = -(aL->second + (bL->second - aL->second) * (real(p) -
        ↪  aL->first) / (bL->first - aL->first));
        return imag(p) <= yu + 1e-9 && imag(p) >= yl - 1e-9;
    } // 53a5a2
};
```

Corte de Polígono

Corta um polígono pelo semiplano à esquerda da reta direcionada $a \to b$ (algoritmo de Sutherland-Hodgman). Encadeie chamadas para clipar por um polígono convexo. Requer $D = double$.

1639c7 - geometry/polygon-clip.cpp - 14 lines

```
// D = double;
// Sutherland-Hodgman clip: keep the left side of directed line a→b.
// Clip poly by half-plane; chain calls for convex clipper.
vector<point> clipPoly(vector<point> &poly, point a, point b) {
    int n = poly.size();
    vector<point> res;
    for (int i = 0; i < n; i++) {
        point c = poly[i], d = poly[(i + 1) % n];
        D cc = cross(b - a, c - a), dc = cross(b - a, d - a);
        if (cc >= 0) res.push_back(c);
        if ((cc > 0) != (dc > 0)) res.push_back(lineIntersect(a, b - a, c, d
        ↪  - c));
    }
    return res;
} // 263dbb
```

Fórmulas Úteis

Contagem e Probabilidade

Número de soluções da equação $x_1 + x_2 + \cdots + x_k = n$ onde $x_i \geq 0$:

$$\binom{n+k-1}{k-1}$$

Número de soluções da equação $x_1 + x_2 + \cdots + x_k = n$ onde $x_i \geq 1$:

$$\binom{n-1}{k-1}$$

Valor esperado do número de tentativas até sucesso com probabilidade p de sucesso:

$$\frac{1}{p}$$

Números de Stirling do primeiro tipo:

$$c(n,k) = \text{Número de permutações de } \{1, 2, \ldots, n\} \text{ com } k \text{ ciclos}$$

$$s(n,k) = (-1)^{n-k} c(n,k)$$

$$x(x-1)(x-2)\ldots(x-n+1) = s(n,1)x + s(n,2)x^2 + \cdots + s(n,n)x^n$$

$$c(n+1,k) = n \times c(n,k) + c(n,k-1), k > 0$$

$$s(n+1,k) = -n \times s(n,k) + s(n,k-1), k > 0$$

Somas

$$\sum_{k=0}^{n} \binom{n}{k} = 2^n$$

$$\sum_{k=1}^{n} k\binom{n}{k} = k2^{n-1}$$

$$c(n, n-k) = \sum_{0 \le i_1 < i_2 < \cdots < i_k < n} i_1 i_2 \ldots i_k$$

$$\sum_{k=0}^{n} c(n,k)x^k = n!\binom{n+x-1}{x-1}, x > 0$$

$$\sum_{k=m}^{n} c(n,k)\binom{k}{m} = c(n+1, m+1)$$

$$\sum_{k=m}^{n} c(n+1, k+1)\binom{k}{m}(-1)^{m-k} = c(n,m)$$

$$\sum_{k=m}^{n} c(k,m)\frac{n!}{k!} = c(n+1, m+1)$$

$$\sum_{k=0}^{n} (m+k)c(m+k, k) = c(n+m+1, n)$$

$$\sum_{k=1}^{n} x^k = 1 + x + \cdots + x^{n-1} = \frac{x^n - 1}{x - 1}$$

$$\sum_{k=1}^{n} kx^{k-1} = 1 + 2x + 3x^2 + \cdots + nx^{n-1} = \frac{nx^{n+1} - ((n+1)x^n) + 1}{(x-1)^2}, x \neq 1$$

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=1}^{n} k^3 = \frac{n^2(n+1)^2}{4}$$

Teoremas e Fatos

- **Teorema do Casamento de Hall.** Um grafo bipartido (L, R) tem um matching saturando L sse

$$\forall S \subseteq L, \quad |N(S)| \ge |S|.$$

- **Teorema de König.** Em um grafo bipartido,

$$\text{matching máximo} = \text{cobertura mínima}.$$

- **Teorema de Dilworth.** Em qualquer poset finito, o tamanho máximo de uma anticadeia é igual ao número mínimo de cadeias necessárias para cobrir todos os elementos.
- **Teorema de Mirsky.** Em qualquer poset finito, o tamanho máximo de uma cadeia é igual ao número mínimo de anticadeias necessárias para cobrir todos os elementos.
- **Condições para trilha/circuito euleriano (direcionado).** Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente fracamente conexa. Então:
  - Existe circuito euleriano sse $\deg^+(v) = \deg^-(v)$ para todo vértice.
  - Existe trilha euleriana sse exatamente um vértice tem $\deg^+(v) = \deg^-(v) + 1$, exatamente um tem $\deg^-(v) = \deg^+(v) + 1$, e todos os outros são balanceados.
- **Condições para trilha/circuito euleriano (não direcionado).** Ignorando os vértices isolados, todos os vértices relevantes devem pertencer a uma única componente conexa. Então:
  - Existe circuito euleriano sse todo vértice tem grau par.
  - Existe trilha euleriana sse exatamente 0 ou 2 vértices têm grau ímpar.
- **Critério de 2-SAT.** Uma instância de 2-SAT é satisfatível sse, para

toda variável x, os literais x e $\neg x$ estão em SCCs diferentes do grafo de implicação.

- **Teorema de Fine–Wilf.** Se uma string tem períodos p e q, e seu tamanho é pelo menos
$$p + q - \gcd(p, q),$$
então ela também tem período $\gcd(p, q)$.
- **Teorema de Pick.** Para um polígono simples na grid,
$$A = I + \frac{B}{2} - 1,$$
onde I é o número de pontos de grid internos e B é o número de pontos de grid na fronteira.
- **Pontos de grid em um segmento.** O número de pontos de grid no segmento de (x_1, y_1) até (x_2, y_2) é
$$\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1.$$
- **Lema de Burnside.** O número de configurações distintas a menos da ação de um grupo finito é
$$\frac{1}{|G|} \sum_{g \in G} \mathrm{fix}(g).$$
- **Fórmula de Cayley.** O número de árvores rotuladas com n vértices é
$$n^{n-2}.$$
- **Fato dos graus no código de Prüfer.** No código de Prüfer de uma árvore rotulada, o vértice v aparece exatamente
$$\deg(v) - 1$$
vezes.
- **Desarranjos.** O número D_n de permutações sem pontos fixos satisfaz
$$D_n = (n-1)(D_{n-1} + D_{n-2}),$$
e também
$$D_n = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!}.$$
- **Teorema de Lucas.** Para primo p, se
$$n = \sum n_i p^i, \qquad k = \sum k_i p^i,$$
então
$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$
- **Fórmula de Legendre.** O expoente de um primo p em $n!$ é
$$v_p(n!) = \sum_{i \geq 1} \left\lfloor \frac{n}{p^i} \right\rfloor.$$
- **Teorema de Kummer.** O expoente de um primo p em $\binom{n}{k}$ é igual ao número de vai-uns ao somar k e $n - k$ na base p.
- **LTE (versão comum).** Se p é um primo ímpar, $p \mid (a - b)$ e $p \nmid ab$, então
$$v_p(a^n - b^n) = v_p(a - b) + v_p(n).$$
Além disso, para n ímpar e $p \mid (a + b)$,
$$v_p(a^n + b^n) = v_p(a + b) + v_p(n).$$
- **Condição geral de compatibilidade do CRT.** O sistema
$$x \equiv a_i \pmod{m_i}$$
tem solução sse, para todos i, j,
$$a_i \equiv a_j \pmod{\gcd(m_i, m_j)}.$$
Quando tem solução, ela é única módulo $\mathrm{lcm}(m_1, \ldots, m_k)$.

- **Inversão de Möbius.** Se

$$g(n) = \sum_{d|n} f(d),$$

então

$$f(n) = \sum_{d|n} \mu(d)\, g\left(\frac{n}{d}\right).$$

- **Princípio da Inclusão–Exclusão.**

$$\left|\bigcup_i A_i\right| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots$$

- **Estrelas e Barras.** O número de soluções inteiras não negativas de

$$x_1 + \cdots + x_k = n$$

é

$$\binom{n+k-1}{k-1}.$$

O número de soluções inteiras positivas é

$$\binom{n-1}{k-1}.$$

- **Números de Catalan.**

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}.$$

- **Teorema de Sprague–Grundy.** Todo jogo imparcial finito em normal play é equivalente a uma pilha de Nim cujo tamanho é seu número de Grundy; o número de Grundy é o mex dos números de Grundy das jogadas possíveis. O número de Grundy de uma soma disjunta é o xor dos números de Grundy das componentes.
- **Erdős–Szekeres.** Toda sequência de $(r-1)(s-1)+1$ números distintos contém ou uma subsequência crescente de tamanho r, ou uma subsequência decrescente de tamanho s.
- **Substrings distintas via suffix array.** Se $lcp[i]$ é o LCP de sufixos adjacentes na ordem do suffix array, então

$$\#\text{substrings distintas} = \frac{n(n+1)}{2} - \sum_i lcp[i].$$

- **Substrings distintas via suffix automaton.** Se $len(v)$ é o maior tamanho do estado v e $link(v)$ é seu suffix link, então

$$\#\text{substrings distintas} = \sum_{v \neq root} \big(len(v) - len(link(v))\big).$$

- **Teorema Matrix-Tree de Kirchhoff.** O número de árvores geradoras de um grafo é igual a qualquer cofator de sua matriz Laplaciana.
- **Existência de raiz primitiva.** Existem raízes primitivas módulo n sse

$$n \in \{1, 2, 4, p^k, 2p^k\},$$

onde p é um primo ímpar.
- **Teorema de Wilson.** Para primo p,

$$(p-1)! \equiv -1 \pmod{p}.$$

- **Inversão binomial.** Se

$$g(n) = \sum_{k=0}^{n} \binom{n}{k} f(k),$$

então

$$f(n) = \sum_{k=0}^{n} (-1)^{n-k} \binom{n}{k} g(k).$$

- **Identidade de Vandermonde.**

$$\sum_k \binom{a}{k}\binom{b}{n-k} = \binom{a+b}{n}.$$

- **Identidade do taco de hockey.**

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}.$$

- **Enumeração de Pólya.** Burnside mais decomposição em ciclos para contar coloracões sob simetria. Em geral, só vale a pena deixar como lembrete se o time realmente usa.
- **Fórmula de Euler planar.** Para um grafo planar conexo,

$$V - E + F = 2.$$

De forma mais geral,

$$V - E + F = 1 + C$$

onde C é o número de componentes conexas.
- **Teorema de Cayley–Hamilton.** Toda matriz quadrada satisfaz seu próprio polinômio característico.
- **Recorrência linear via polinômio característico.** Se uma sequência é gerada por multiplicações repetidas por uma matriz $k \times k$, então ela satisfaz uma recorrência linear de ordem no máximo k.
- **Desigualdade de Monge.** Um array A é Monge se

$$A[i][j] + A[i'][j'] \leq A[i][j'] + A[i'][j] \qquad (i \leq i',\ j \leq j').$$

Essa é a condição estrutural por trás de várias otimizações de DP.
- **Condição da otimização de Knuth (forma de lembrete).** Para DP de intervalos,

$$opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$$

sob as hipóteses usuais de desigualdade do quadrilátero / monotonicidade.
- **Condição da otimização de DP por divide and conquer (forma de lembrete).** Se o índice minimizante é monótono,

$$opt[i][j] \leq opt[i][j+1],$$

então cada camada pode ser calculada com otimização por divide and conquer.
- **Soma de Minkowski de polígonos convexos.** A soma de Minkowski de dois polígonos convexos pode ser calculada em tempo linear após ordenar ciclicamente pelas direções das arestas.
- **Teorema de Helly em $\mathbb{R}^2$.** Para uma família finita de conjuntos convexos no plano, se todo subconjunto de 3 deles intersecta, então todos intersectam.
- **Teorema de Helly em $\mathbb{R}^d$.** Para conjuntos convexos em $\mathbb{R}^d$, se todo subconjunto de $d+1$ deles intersecta, então todos intersectam.
- **Teorema de Carathéodory.** Se um ponto pertence ao fecho convexo de um conjunto em $\mathbb{R}^d$, então ele pertence ao fecho convexo de no máximo $d+1$ pontos desse conjunto.
- **Teorema de Radon.** Qualquer conjunto com pelo menos $d+2$ pontos em $\mathbb{R}^d$ pode ser particionado em dois subconjuntos disjuntos cujos fechos convexos se intersectam.

Ideias

Gerais

- Em problemas envolvendo somas em intervalos de arrays, pode ser mais fácil trabalhar com o array de somas de prefixos.
- Muitas DPs de bitmask podem ser simplificadas para usar apenas alguma propriedade da bitmask. (Número de bits, etc)
- Se o problema descreve um processo que parece complicado, talvez seja mais fácil ver o processo ao contrário.

Calculando somas e integrais

- Verificar se existe uma fórmula fechada
- Inverter a ordem da soma (em somatórios duplos)
- Transformar em somatório duplo e inverter a ordem
- Verificar se a soma pode ser calculada de um jeito eficiente (sem ser uma fórmula fechada)

- No caso de calcular várias somas, verificar se existe uma relação entre elas (Calcular uma pode ajudar a calcular outra)

Problemas de Otimização

- Encontrar um upper bound e provar que ele sempre é alcançável.
- Encontrar uma construção boa, e provar que não é possível fazer melhor.
- Escrever um brute-force e testar com casos pequenos para encontrar algum padrão.

Debugging

Geral

- Trocou nomes de variáveis? (m por n, j por i, ...)
- Esqueceu algum caso especial? ($n = 1$, $m = 0$, ...)
- Esqueceu de trocar entre 0-based e 1-based?

WA - Wrong Answer

- Esqueceu o `define int i64`?
- Esqueceu algum caso base?
- verificar se o valor inicial da DP está correto.

RE - Runtime Error

- Verificar asserts.
- Verificar limites de vectors.
- Verificar se `maxn` é o suficiente.